



US 20250278404A1

(19) **United States**

(12) **Patent Application Publication**
Kepten et al.

(10) **Pub. No.: US 2025/0278404 A1**

(43) **Pub. Date: Sep. 4, 2025**

(54) **SEPARATED DATABASE MANAGEMENT**

(71) Applicant: **Pliops Ltd.**, Tel Aviv (IL)

(72) Inventors: **Omer Kepten**, Haifa (IL); **Michael Pan**, Tel Aviv (IL); **Moshe Twitto**, Givat Shemuel (IL); **Edward Bortnikov**, Tel Aviv (IL)

(73) Assignee: **Pliops Ltd.**, Tel Aviv (IL)

(21) Appl. No.: **18/595,351**

(22) Filed: **Mar. 4, 2024**

Publication Classification

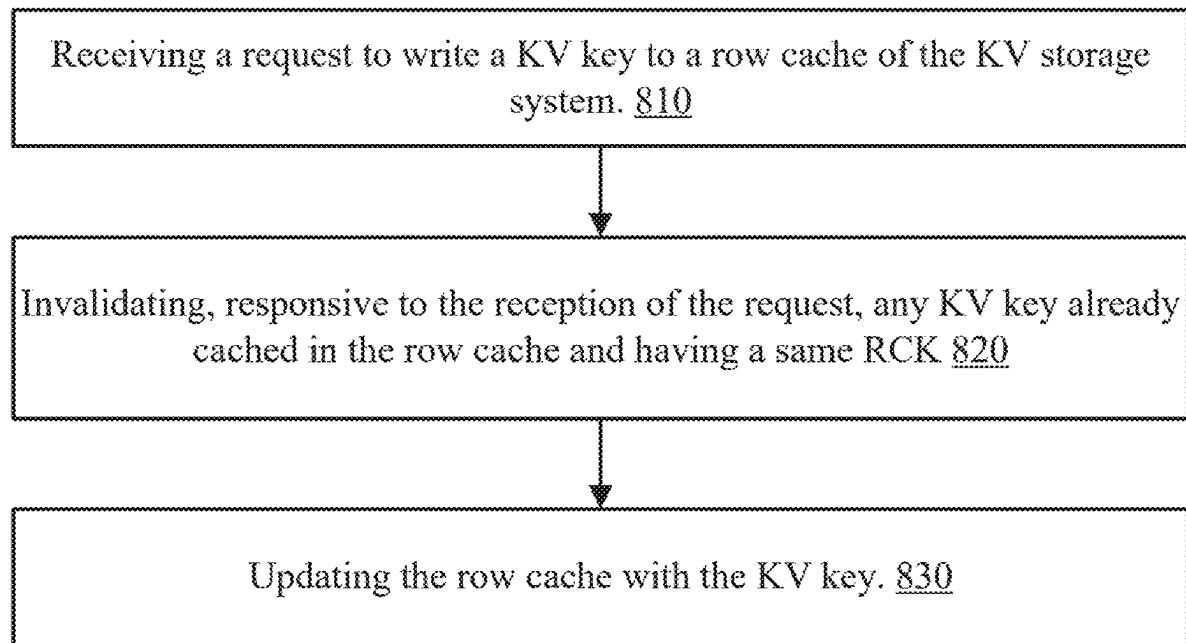
(51) **Int. Cl.**
G06F 16/2455 (2019.01)

(52) **U.S. Cl.**

CPC **G06F 16/24552** (2019.01)

(57) **ABSTRACT**

A method for updating a key-value (KV) storage system, the method includes (i) receiving a request to write a KV key to a row cache of the KV storage system, wherein the request is associated with a row cache key (RCK), wherein the RCK comprises a row cache identifier (RCID) and a column family identifier (CFID), wherein the KV storage system further comprises a separated database, the separated database comprises a log structured merge (LSM) tree and KV database that is separated from LSM tree; and (ii) invalidating, responsive to the reception of the request, any KV key already cached in the row cache and associated with a same RCK as the RCK associated with the request.



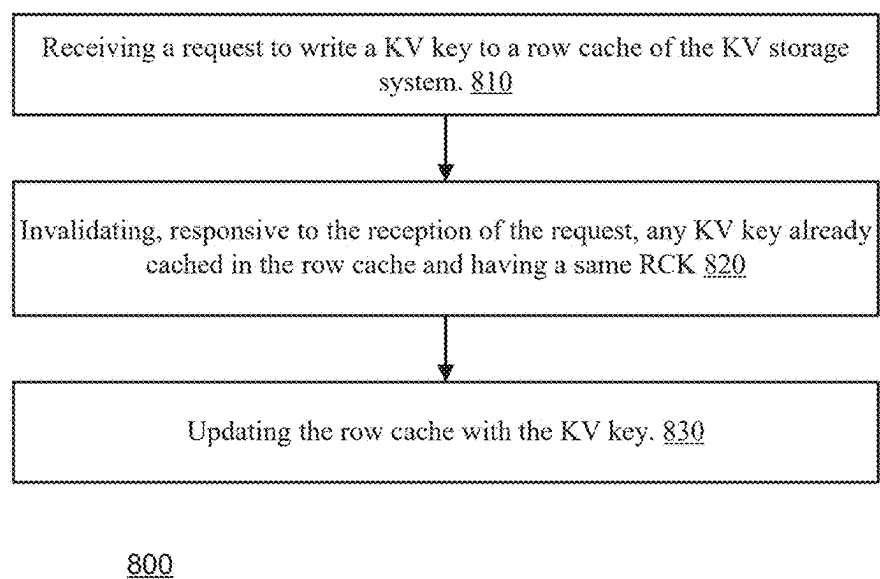


FIG. 1A

RCID <u>911</u>	CFID <u>912</u>	KV key <u>919</u>
-----------------	-----------------	-------------------

910

RCID <u>911</u>	CFID <u>912</u>	SID <u>914</u>	KV key <u>919</u>
-----------------	-----------------	----------------	-------------------

920

RCID <u>911</u>	CFID/FID flag <u>915</u>	CFID/FID <u>916</u>	KV key <u>919</u>
-----------------	-----------------------------	------------------------	-------------------

930

RCID <u>911</u>	CF/FID flag <u>915</u>	CFID/FID <u>916</u>	SID <u>914</u>	KV key <u>919</u>
-----------------	---------------------------	------------------------	----------------	-------------------

940

FIG. 1B

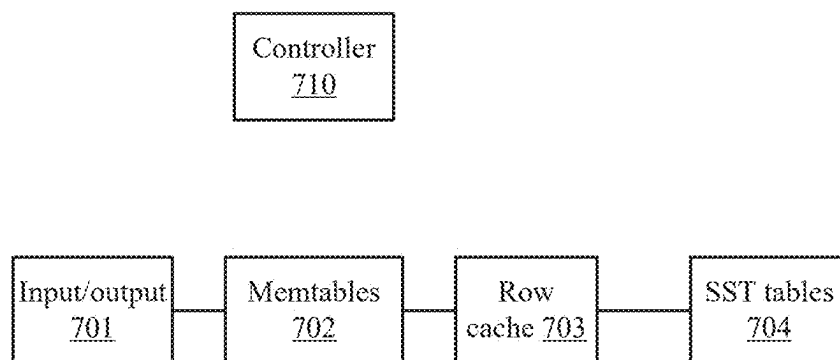


FIG. 1C

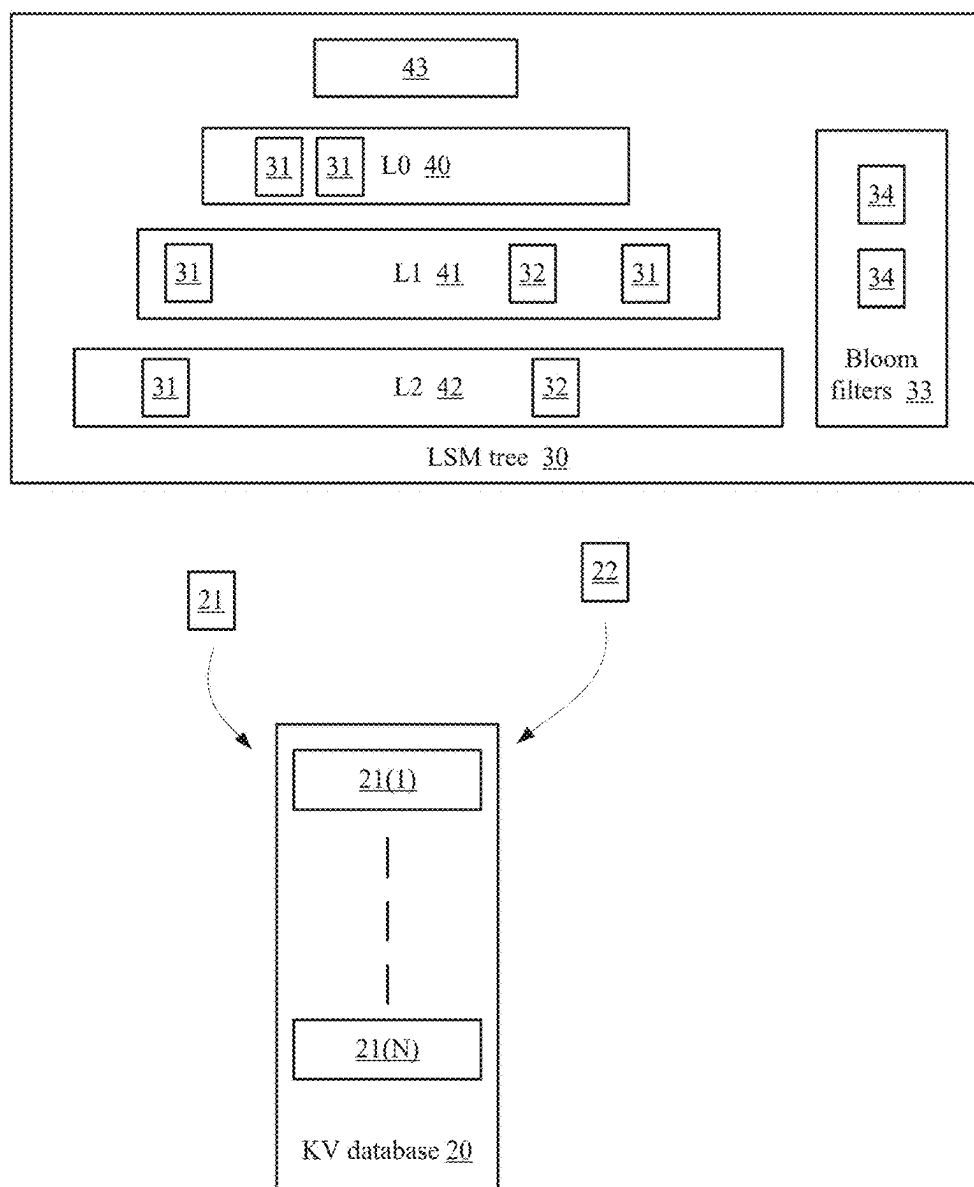


FIG. 2A

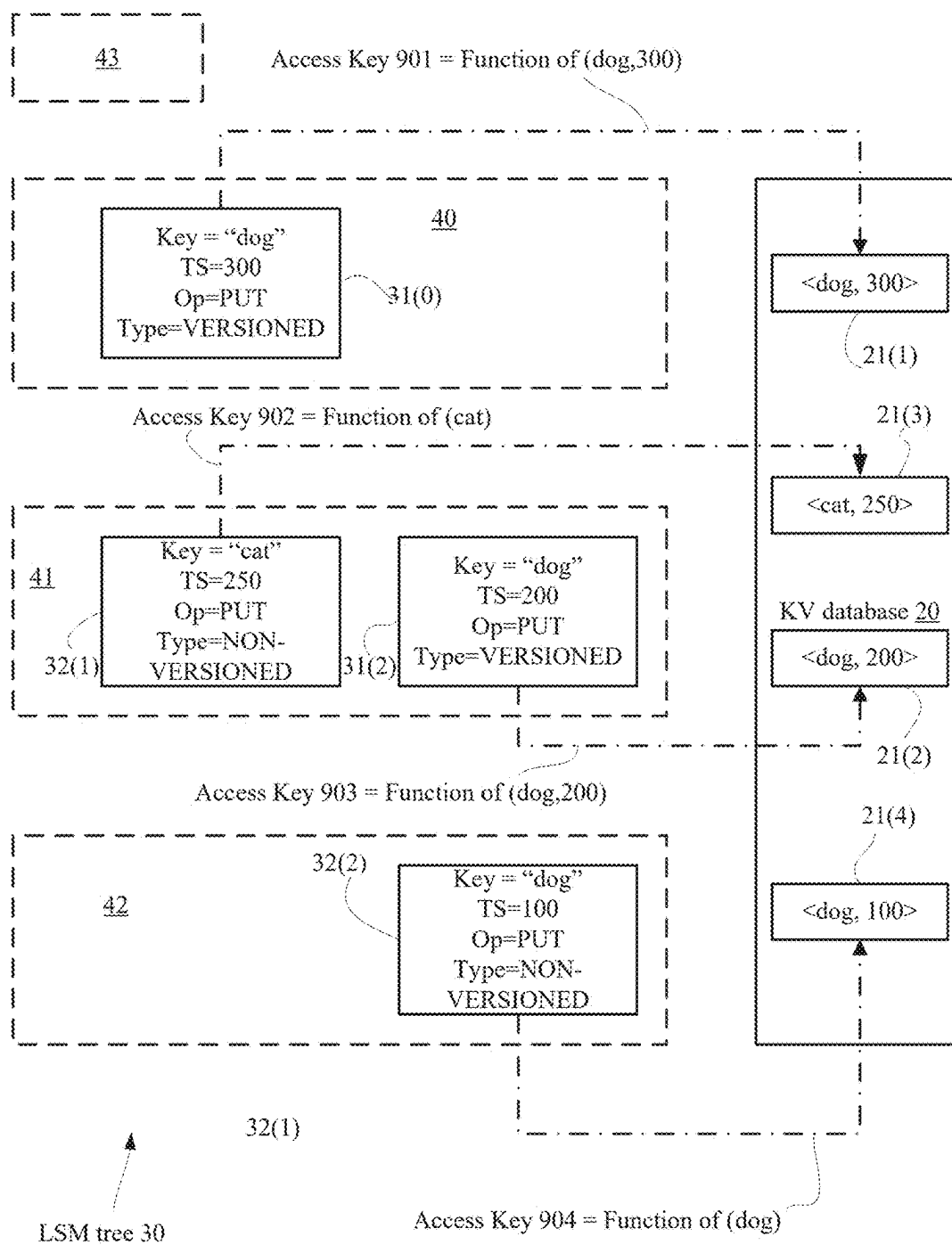
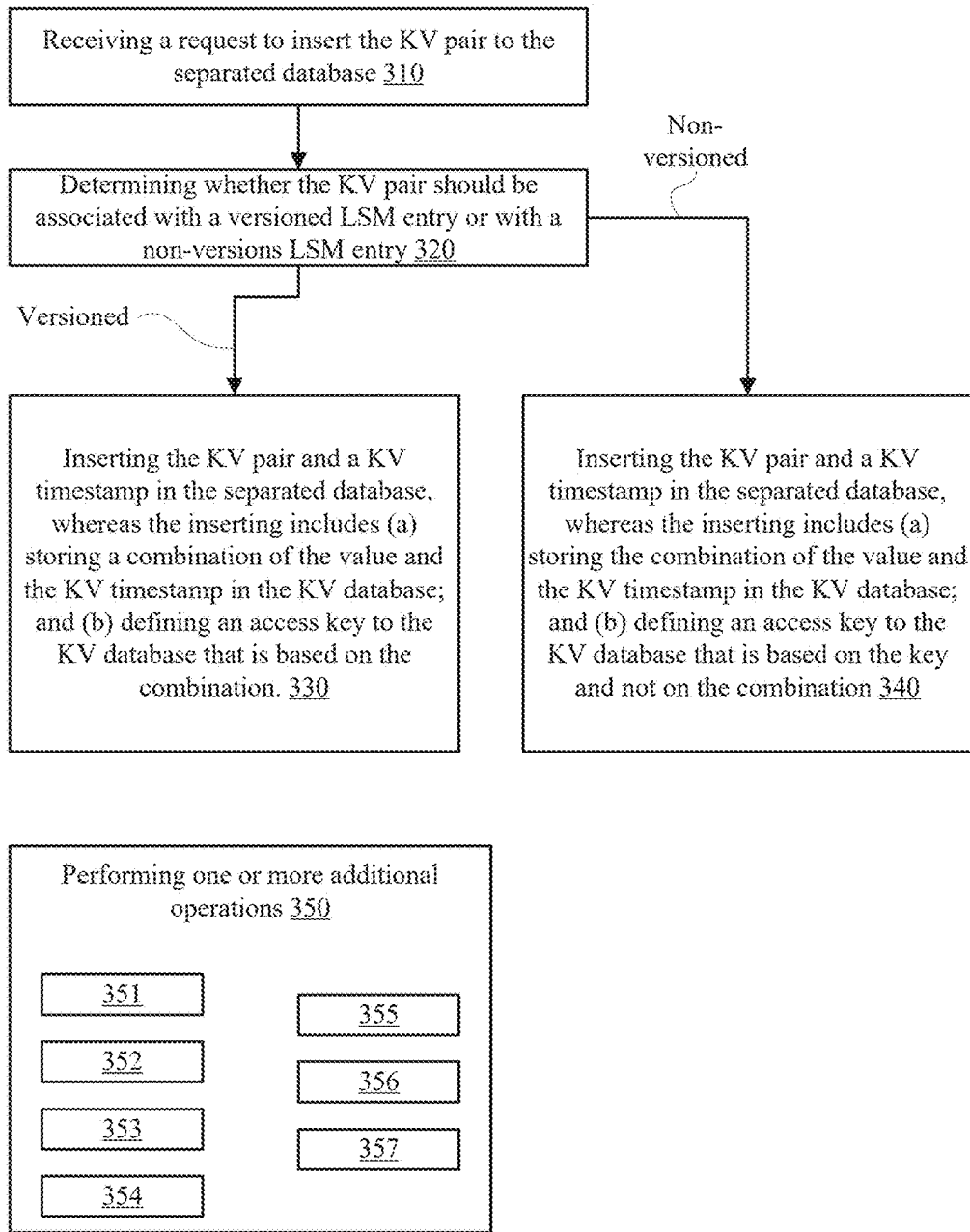


FIG. 2B



300

FIG. 3

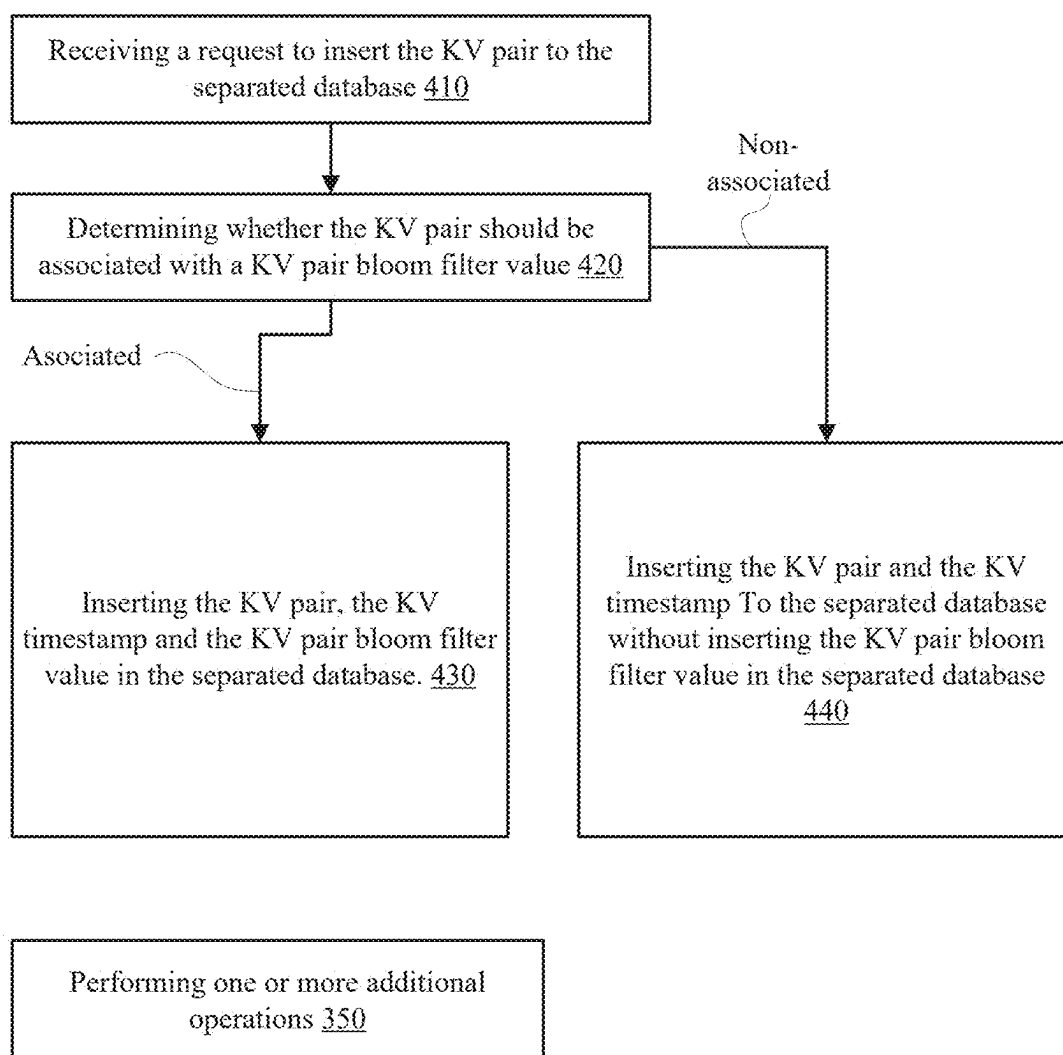
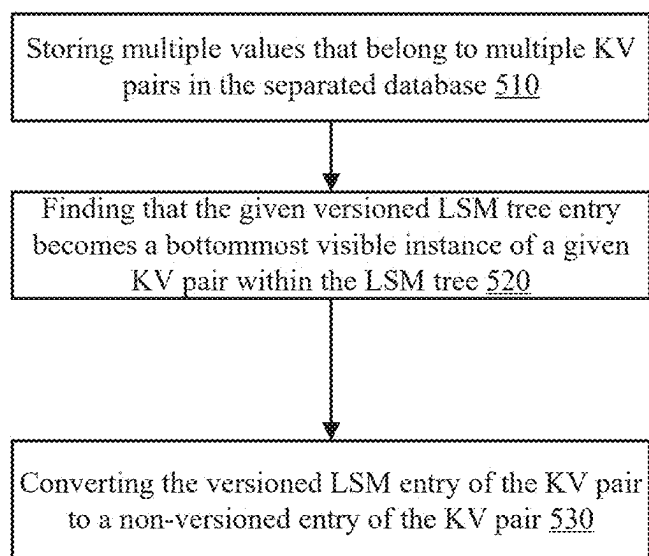
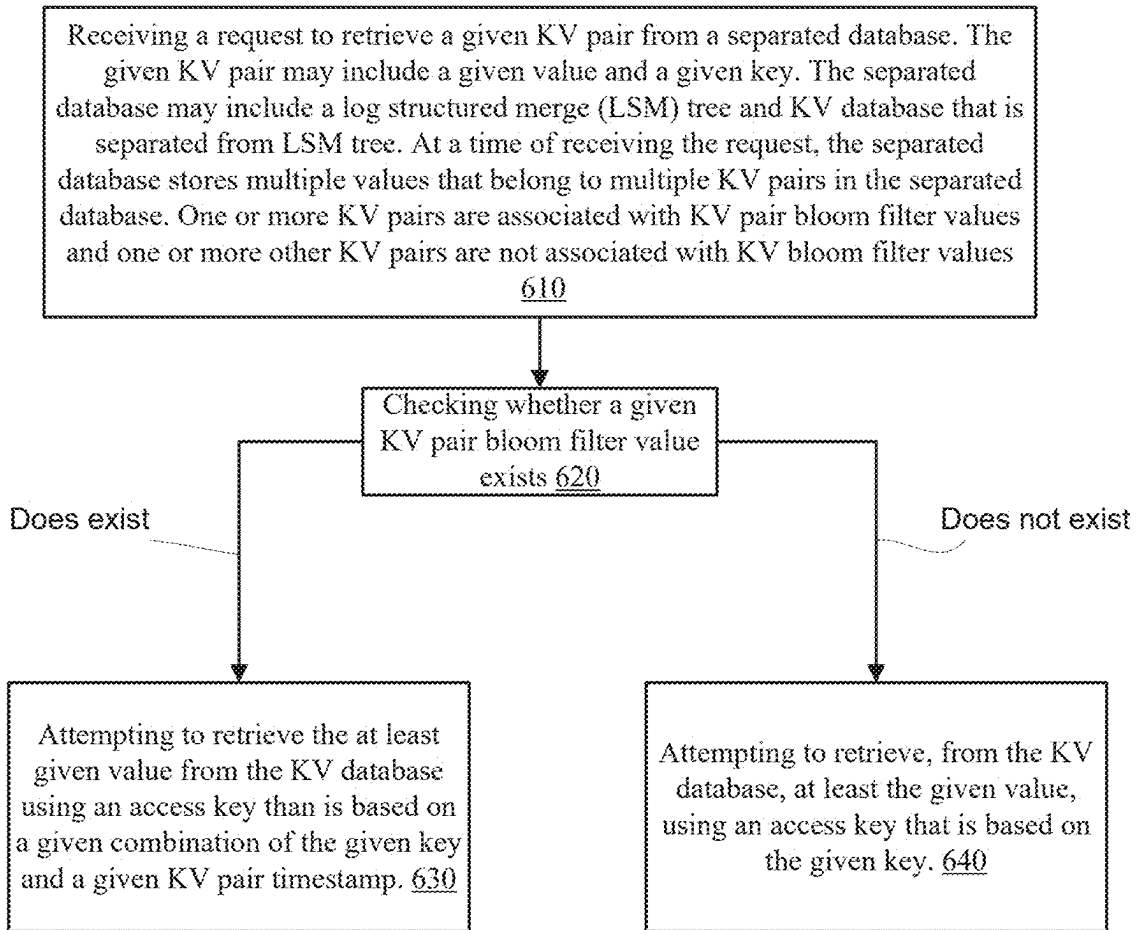
400

FIG. 4



500

FIG. 5



600

FIG. 6

SEPARATED DATABASE MANAGEMENT

BACKGROUND

[0001] A log structured merge (LSM) tree is a data structure that iteratively applies merge operations—files of one layer are merged and are moved to the next later.

[0002] Various storage systems include cache memories such as a row cache. The row cache caches content that is accesses using a row cache key that includes a file identifier that identifies the file associated with the content.

[0003] The merge operations executed in relation to the LSM tree causes the row cache key to be of short duration—resulting in an inefficient usage of the row cache.

[0004] There is a growing need to provide a solution that improve the usage of the row cache.

BRIEF DESCRIPTION OF THE DRAWINGS

[0005] The subject matter that is regarded as the invention is particularly pointed out and distinctly claimed in the concluding portion of the specification. The invention, however, both as to organization and method of operation, together with objects, features, and advantages thereof, may best be understood by reference to the following detailed description when read with the accompanying drawings in which:

[0006] FIG. 1A illustrate an example of a method;

[0007] FIG. 1B illustrates examples of row cache keys;

[0008] FIG. 1C illustrates an example of various components of a storage system;

[0009] FIG. 2A illustrates an example of a separated database;

[0010] FIG. 2B illustrates an example of a separated database;

[0011] FIG. 3 illustrates an example of a method;

[0012] FIG. 4 illustrates an example of a method;

[0013] FIG. 5 illustrates an example of a method; and

[0014] FIG. 6 illustrates an example of a method.

DETAILED DESCRIPTION OF THE DRAWINGS

[0015] There is provided a method for improving a usage of a row cache of a key value storage system that includes a separated data structure.

[0016] The benefits of having a separated data structure are listed in other parts of the detailed description.

[0017] The improvement of the usage of the row cache includes expanding the relevancy window of content stored in the row cache while preventing access errors resulting from having row cache key that do not depend on a file identifier.

[0018] According to an embodiment, a row cache is provided that incorporates a row cache key (RCK) composed of the row cache identifier (may be similar to a database identifier), a column family identifier, and a KV key.

[0019] According to an embodiment, a PUT request associated with the RCK or subsequent PUT operations on that KV key may invalidate the corresponding row cache entry, guaranteeing that hits in the row cache always yield valid data. A DELETE request associated with the KV key will also invalidate the related row cache entry.

[0020] According to an embodiment, no distinction is made between versioned and non-versioned KV keys in relation to the row cache—which results in better utilization)

[0021] According to an embodiment, on a Get() with a snapshot, the KV key will be entered into the row cache with a RCK that includes a snapshot identifier (for example a snapshot number) regardless of whether the current entry in the row cache is valid to that snapshot. Going forward, any Get() with that snapshot will hit.

[0022] According to an embodiment, if the same version of the KV key is relevant for two different snapshots, it may require two different entries in the row cache.

[0023] According to an embodiment, the suggested solution increases row cache entry lifetime than a solution which is dependent on a file ID—and especially to such a solution that concatenates the snapshot to the RCK.

[0024] FIG. 1A illustrates an example of method **800** for updating a key-value (KV) storage system. A KV storage system is a storage system configured to store KV pairs.

[0025] According to an embodiment, method **800** includes step **810** of receiving a request (for example a PUT access request) to write a KV key to a row cache of the KV storage system.

[0026] The request is associated with a row cache key (RCK), wherein the RCK includes a row cache identifier (RCID) and a column family identifier (CFID).

[0027] A column family is a database object that contains columns of related data. Users may define different column families and store the different columns families in the KV storage system. The CFID is used to differentiate between different columns families.

[0028] According to an embodiment, there are different row caches that are accessible to one or more KV storage systems—and the RCID is used to differentiate between one row cache to another.

[0029] The KV storage system further stores a separated database. The separated database includes an LSM tree and a KV database that is separated from LSM tree. An example of a separated database is illustrated in U.S. patent application Ser. No. 17/651,065 which is incorporated herein by reference.

[0030] According to an embodiment, step **810** is followed by step **820** of invalidating, responsive to the reception of the request, any KV key that is already cached in the row cache, associated with the same RCK.

[0031] According to an embodiment, step **820** is followed by step **830** of updating the row cache with the KV key.

[0032] According to an embodiment, step **810** includes (or is followed by or is preceded by) storing the KV key at another memory—such as a memory table (that may belong to memory tables—known as Memtables).

[0033] According to an embodiment, step **830** is triggered after a reception of another access request (for example a GET access request) that resulted in a row cache miss.

[0034] According to an embodiment, step **830** is triggered after a reception of another access request that resulted in a combination of a row cache miss and a Memtables miss.

[0035] According to an embodiment, step **830** is triggered following the invalidation—for example—as a result of the PUT access request.

[0036] According to an embodiment, an RCK includes a snapshot identifier during a read operation. According to an embodiment, an RCK does not include a snapshot identified when performing a delete or a write (PUT) access request.

[0037] According to an embodiment, the RCK does not include a file identifier (FID) that is indicative of a LSM file associated with the KV key.

[0038] According to an embodiment, step 820 is executed without evaluating a timing relationship between the KV key and any of the KV keys already cached in the row cache. As the RCK does not provide an indication of which file is associated with the KV pair and there is no clear indication within the RCK to the source of the KV pair—except of the column family identifier—the currently received KV pair may be received out of order and may be an older version of a KV pair currently stored in the row cache—the invalidation guarantee that the received KV pair—following the invalidation—is the most current KV pair stored in the cache row.

[0039] According to an embodiment, the KV key is a versioned KV key.

[0040] According to an embodiment, the KV key is a non-versioned KV key.

[0041] An example of a storage system (KV storage system) that supports a separated database, and of versioned and non-versioned keys is provided. Method 800 may be implemented by amending the storage system.

[0042] FIG. 1B illustrates a first example of a first RCK 910 that does not refer to a snapshot.

[0043] RCK 910 includes row cache identifier (denoted RCID 911) and a column family identifier (denoted CFID 912). RCK 910 may also include KV key 919—when there is a request to write the KV key.

[0044] FIG. 1B illustrates a second example of a second RCK 920 that refers to a snapshot.

[0045] RCK 920 includes RCID 911, CFID 912 and snapshot identifier (SID 914). RCK 920 may also include KV key 913.

[0046] FIG. 1B illustrates a third example of a third RCK 930 that does not refer to a snapshot and may include a file identifier (FID) or the CFID. In order to facilitate both options—RCK 930 includes an FID or CFID flag (denoted CFID/FID flag 915) that indicates whether the value in CFID/DIF field 916 is a FID or a CFID. RCK 930 may also include KV key 919.

[0047] FIG. 1B illustrates a third example of a fourth RCK 940 that does refer to a snapshot and may include a FID or a CFID. In order to facilitate both options—RCK 940 includes CFID/FID flag 915, CFID/DIF field 916, and SID 914. RCK 940 may also include KV key 919.

[0048] FIG. 1C illustrates an example of various components of a KV storage system—that include:

[0049] a. An input/output unit 710 is configured to receive access requests and output responses to the access requests.

[0050] b. Memory tables (Memtables) 702 that form a memory buffer such as memory buffer 43 of FIG. 2A.

[0051] c. Row Cache 703. The row cache 703 is larger than the memtables 702.

[0052] d. Sorted String Tables (SSTs) 704 that form the LSM tree that include KV keys that associated with KV pairs stored in the KV database.

[0053] e. Controller 710 for controlling the execution of the KV storage system. The controller may include one or more integrated circuits, may be a part of an integrated circuit.

[0054] According to an embodiment, there is provided a method for updating a key-value (KV) storage system, the method includes (i) receiving a request to write a KV key to a row cache of the KV storage system, wherein the request is associated with a row cache key (RCK), wherein the RCK includes a row cache identifier (RCID) and a column family

identifier (CFID), wherein the KV storage system further includes a separated database, the separated database includes a log structured merge (LSM) tree and KV database that is separated from LSM tree; and (ii) invalidating, responsive to the reception of the request, any KV key already cached in the row cache and having a same value as a value of the KV key.

[0055] According to an embodiment, the invalidating is executed without evaluating a timing relationship between the KV key and any of the KV keys already cached in the row cache.

[0056] According to an embodiment, the RCK does not include a file identifier (FID) that is indicative of a LSM file associated with the KV key.

[0057] According to an embodiment, the KV key is associated with a snapshot and the RCK includes a snapshot identifier.

[0058] According to an embodiment, the KV key is a versioned KV key.

[0059] According to an embodiment, the KV key is a non-versioned KV key.

[0060] According to an embodiment, method further includes updating the row cache with the KV key.

[0061] According to an embodiment, the updating is triggered by a reception, by the KV storage system, of a request to read the KV pair.

[0062] According to an embodiment, the updating is triggered by a reception, by the KV storage system, of a request to read the KV pair that resulted in a memory tables related miss, wherein the memory tables store the LSM tree.

[0063] According to an embodiment, the updating is triggered by a reception, by the KV storage system, of a request to read the KV pair that resulted in a memory tables related miss and in a row cache miss, wherein the memory tables store the LSM tree.

[0064] According to an embodiment, there is provided a non-transitory computer readable medium for updating a key-value (KV) storage system, the non-transitory computer readable medium stores instructions for: (i) receiving a request to write a KV key to a row cache of the KV storage system, wherein the request is associated with a row cache key (RCK), wherein the RCK includes a row cache identifier (RCID) and a column family identifier (CFID), wherein the KV storage system further includes a separated database, the separated database includes a log structured merge (LSM) tree and KV database that is separated from LSM tree; and (ii) invalidating, responsive to the reception of the request, any KV key already cached in the row cache and having a same value as a value of the KV key.

[0065] According to an embodiment, the invalidating is executed without evaluating a timing relationship between the KV key and any of the KV keys already cached in the row cache.

[0066] According to an embodiment, the RCK does not include a file identifier (FID) that is indicative of a LSM file associated with the KV key.

[0067] According to an embodiment, the KV key is associated with a snapshot and the RCK includes a snapshot identifier.

[0068] According to an embodiment, the KV key is a versioned KV key.

[0069] According to an embodiment, the KV key is a non-versioned KV key.

[0070] According to an embodiment, non-transitory computer readable medium further stores instructions for updating the row cache with the KV key.

[0071] According to an embodiment, the updating is triggered by a reception, by the KV storage system, of a request to read the KV pair.

[0072] According to an embodiment, the updating is triggered by a reception, by the KV storage system, of a request to read the KV pair that resulted in a memory tables related miss, wherein the memory tables store the LSM tree.

[0073] According to an embodiment, there is provided a key-value (KV) storage system, includes: a controller that is configured to receive a request to write a KV key to a row cache of the KV storage system, wherein the request is associated with a row cache key (RCK), wherein the RCK includes a row cache identifier (RCID) and a column family identifier (CFID), wherein the KV storage system further includes a separated database, the separated database includes a log structured merge (LSM) tree and KV database that is separated from LSM tree; and invalidate, responsive to the reception of the request, any KV key already cached in the row cache and having a same value as a value of the KV key.

[0074] According to an embodiment, the invalidating is executed without evaluating a timing relationship between the KV key and any of the KV keys already cached in the row cache.

[0075] According to an embodiment, the RCK does not include a file identifier (FID) that is indicative of a LSM file associated with the KV key.

[0076] According to an embodiment, the KV key is associated with a snapshot and the RCK includes a snapshot identifier.

[0077] According to an embodiment, the KV key is a versioned KV key.

[0078] According to an embodiment, the KV key is a non-versioned KV key.

[0079] According to an embodiment, the controller that is configured to update the row cache with the non-versioned KV key.

[0080] According to an embodiment, the updating is triggered by a reception, by the KV storage system, of a request to read the KV pair.

[0081] According to an embodiment, the updating is triggered by a reception, by the KV storage system, of a request to read the KV pair that resulted in a memory tables related miss, wherein the memory tables store the LSM tree.

[0082] Following description provides an examples of the KV storage system, a separated database and versioned and non-versioned keys.

[0083] The storage system supports a separated database that may support storage engine APIs such as (a) random write—put, delete, multiput (put/delete batch), (b) random read—get, multiget, (c) range scan—ordered iteration, and (d) range delete.

[0084] The separated database may be optimized for high throughput, low tail latency of (a) random write path—low write amplification (for example—close to 2, including WAL), and (b) random read path (get/multiget)—low read amplification (for example—close to 1).

[0085] The separated database may provide best-case performance achieved when the fraction of same-key overwrites updates is small.

[0086] The separated database may have (or may be accessed by) batch (multi-* and range-*) API's that are atomic.

[0087] The multi get and range scan may provide snapshot semantics, regardless of whether they are invoked via a snapshot or not. In the latter case, the implementation defines a transient snapshot.

[0088] The method implements key-value separation, to avoid write amplification upon compaction. In other words, the keys reside in the LSM tree, whereas the values reside in the external KV store. For simplicity of presentation, we assume that all KV pairs are separated, some values may be stored in the LSM tree (e.g., small values). The LSM tree entries are therefore called references (to objects stored in KV).

[0089] An LSM entry is a version of some change that happened to the KV pair, with a unique timestamp (KV pair timestamp).

[0090] There may be two kinds of entries: a PUT entry (standing either for an insert or for an update) and a DELETE entry (standing for a deletion, also named tombstone).

[0091] A KV entry may include a combination of a key (of a KV pair) and a timestamp of its LSM reference (KV pair timestamp).

[0092] A read access to the KV entry requires two I/O operations: (a) one to the LSM tree to retrieve the reference, and one more to the KV database. This doubles the get latency. There is a need to have the tail get latency very close to that of the KV.

[0093] The suggested method may optimize the get latency for those KV pairs that did not undergo multiple in-place updates.

[0094] The method may utilize a special LSM PUT entry type, named a non-versioned entry (of an LSM tree).

[0095] A non-versioned LSM tree entry may have two properties: (a) have their lookup in the KV database is by an access key that is based on the key and not on a combination of the key and the KV pair timestamp (<key,timestamp>), and (b) they do not have a KV pair bloom filter value—such value is not stored in a SST file Bloom filters.

[0096] Any reference to an access key to the KV database means an access key for retrieving at least a value of a KV pair.

[0097] The method may maintain the invariant that at most one visible non-versioned entry exists for each key. (An entry is called invisible if it cannot be accessed by any get or scan). Reminder—entries become invisible when shadowed by more recent updates to the key).

[0098] The combination may be stored in the KV database in a concatenated form.

[0099] A timestamp related to a non-versions LSM entry may be used for verification. The verification is required because the access key it the value and does not include the KV pair timestamp. The verification may include checking of the retrieved value complies with any time restriction related to a read request—for example having a timestamp that does not exceed a snapshot mentioned in the read request.

Get Method (Sketch)

[0100] Every visible version in the LSM tree is accessible, in one of two ways: (A)—the LSM tree search (by key) leads to a “normal” versioned entry. The ensuing lookup in the KV

database using a combination ($\langle \text{key}, \text{timestamp} \rangle$) based access key. (B)—the LSM tree search fails to find if the key exists via the Bloom filter search. This still leaves room for existence of a non-versioned entry, therefore we must follow up and search in the KV database using an access key that is based on the key and not the combination.

[0101] The correctness of this access scheme may depend on the invariant that at most one visible entry per key is non-versioned.

[0102] The second search path does not require any I/O to access the LSM tree, assuming that all the Bloom filters are small, and get cached. If most KV pairs have a single version, then most get's entail one I/O, except the false positives, which entail an extra lookup in the LSM tree.

[0103] The scan path does not use Bloom filters. It directly de-references each entry in the range (either versioned or non-versioned).

Flush and Compaction Method (Sketch)

[0104] The method maintains the “single-visible-non-versioned-entry” property. We choose to maintain the earliest entry in the LSM tree for each key as non-versioned. In the LSM tree structure, this is the bottommost entry (i.e., there are no visible entries at the larger levels).

[0105] There may be two conditions to check that the entry is eligible to become a non-versioned reference: (a) there is no active snapshot with a timestamp smaller than the entry's (i.e., that the access to this snapshot might need earlier versions), and (b) the key is not represented in any Bloom filter of the larger (lower) levels.

[0106] Note that in the lowest level, there may be multiple versions due to live snapshots.

[0107] The flush method may include using the memory buffer (memtable) as a source of all entries. Every PUT entry becomes either versioned or non-versioned. This defines under which access key the object is inserted into KV database. In the first case, it is the combination $\langle \text{key}, \text{timestamp} \rangle$, and the value that represent the key is added to the new SST file's Bloom filter. In the second case, the access key is based on $\langle \text{key} \rangle$, and a value representative of the key is not added to the Bloom filter. A bloom filter may be provided per run, per LSM layer and the like. Note that the latter case may entail an overwrite of an object under the same key that stands for a version that is not visible anymore.

[0108] The compaction method, which merges multiple SST files, may be similar to flush, except for the case of “renaming” a PUT entry from versioned to non-versioned. When a versioned entry becomes bottommost visible (i.e., the previous bottommost visible entry becomes obsolete), its referenced value is re-inserted into KV database under access key $\langle \text{key} \rangle$, and the old value is deleted. As before, the key is not added to the generated Bloom filter.

[0109] A race condition can emerge when two new SST files are flushed in parallel. In that case, the file that stands for a younger memtable may not use the Bloom filter of the second file, because the latter has not been finalized yet. For that reason, the method disables concurrent flushes into the same LSM tree.

[0110] The method may handle a potential race condition between rename (versioned $\rightarrow$ non-versioned transition) and non-versioned entry deletion, for the same key, in two concurrent compactions.

Environment

[0111] FIG. 2A illustrates an example of a separated database 10 that includes a KV database 20 and an LSM tree 30. LSM tree 30 is illustrated as including three levels L0 40, L1 41 and L2 42, and a memory buffer 43. The LSM tree 30 may include two levels or more than three levels.

[0112] The KV database 20 may store combinations 21(1)-21(N) of $\langle \text{value}, \text{KV pair timestamp} \rangle$. The KV database 20 may be accessed by access keys—that may include first access keys 21 that are based on the combinations, and second access keys 22 that may be based on the keys and not on the entire combinations.

[0113] The LSM tree may include versioned LSM entries 31, non-versioned entries 32 and Bloom filters 33 that may include block filter KV pair values 34.

[0114] FIG. 2B illustrates an example of a separated database 10 in which:

[0115] L0 includes a versioned LSM entry 31(0) in which the key is “dog”, the KV pair timestamp is 300, and the access type is PUT. The versioned LSM entry 31(0) includes a VERSIONED indicator. The access key 901 to combination 21(1) of $\langle \text{dog}, 300 \rangle$ within the KV database is based on $\langle \text{dog}, 300 \rangle$.

[0116] L1 includes a versioned LSM entry 31(1) in which the key is “dog”, the KV pair timestamp is 200, and the access type is PUT. The versioned LSM entry 31(1) includes a VERSIONED indicator. The access key 902 to combination 21(2) of $\langle \text{dog}, 200 \rangle$ within the KV database is based on $\langle \text{dog}, 200 \rangle$.

[0117] L1 also includes a non-versioned LSM entry 32(1) in which the key is “cat”, the KV pair timestamp is 250, and the access type is PUT. The non-versioned LSM entry 32(1) includes a NON-VERSIONED indicator. The access key 903 to combination 21(3) of $\langle \text{cat}, 250 \rangle$ within the KV database based on cat.

[0118] L2 includes a non-versioned LSM entry 32(2) in which the key is “dog”, the KV pair timestamp is 100, and the access type is PUT. The non-versioned LSM entry 32(2) includes a NON-VERSIONED indicator. The access key 904 to the KV database based on dog.

[0119] FIG. 3 illustrates method 300 for inserting a key value (KV) pair to a separated database.

[0120] Method 300 may start by step 310 of receiving a request to insert the KV pair to the separated database. The request may be a PUT command. The separated database may include a log structured merge (LSM) tree and KV database that is separated from LSM tree.

[0121] Step 310 may be followed by step 320 of determining whether the KV pair should be associated with a versioned LSM entry or with a non-versions LSM entry.

[0122] Step 320 may include maintaining up to a single visible non-versioned LSM entry for a single KV pair.

[0123] Step 320 may include checking whether the separated database currently stores a visible other instance of the KV pair. If the answer is positive-then the KV pair should be associated with a versioned LSM tree entry.

[0124] When determining that the KV pair should be associated with a versioned LSM entry then step 320 may be followed of step 330 of inserting the KV pair and a KV timestamp in the separated database, whereas the inserting includes (a) storing a combination of the value and the KV timestamp in the KV database; and (b) defining an access key to the KV database that is based on the combination.

[0125] When determining that the KV pair should be associated with a non-versioned LSM entry then step 320 may be followed of step 340 of inserting the KV pair and a KV timestamp in the separated database, whereas the inserting includes (a) storing the combination of the value and the KV timestamp in the KV database; and (b) defining an access key to the KV database that is based on the key and not on the combination.

[0126] Step 330 may include adding a KV pair bloom filter value when determining that the KV pair should be associated with the versioned LSM entry then.

[0127] Step 340 may include preventing from adding a KV pair bloom filter value when determining that the KV pair should be associated with the non-versioned LSM entry. Step 340 may include preventing from calculating the KV pair bloom filter value.

[0128] The LSM tree and bloom filters associated with the LSM tree may be stored in one or more local memory units (for example cached, stored in SSDs), and the KV database may be stored in one or more remote memory units.

[0129] Method 300 may include step 350 of performing one or more additional operations. The one or more additional operations may include a memory management operation (for example flushing, compaction) and/or responding to requests such as read requests.

[0130] Step 350 may include at least one of the following:

[0131] Step 351 of performing a flushing operation.

[0132] Step 352 of performing a compaction operation.

[0133] Step 353 of converting a versioned LSM entry of the KV pair to a non-versioned entry of the KV pair when the versioned LSM entry becomes a bottommost visible instance of the KV pair within the LSM tree.

[0134] Step 354 of performing a compaction operation of the LSM tree while preventing from a deletion of a LSM entry that will become obsolete at a completion of the compaction operation till an end of the compaction operation.

[0135] Step 355 of preventing parallel flushes operations from a write buffer of the LSM tree to a non-volatile stored layer of the LSM tree.

[0136] Step 356 of receiving a request to retrieve the KV pair from the separated database; checking whether a KV pair bloom filter value exists and when the KV pair bloom filter value does not exist then attempting to retrieve the combination using the access key that is based on the key. The attempting may include accessing the KV database to retrieve a combination. The combination may be verified using the KV pair timestamp of the combination.

[0137] Step 357 of receiving a request to retrieve the KV pair from the separated database; checking whether a KV pair bloom filter value exists; and when the KV pair bloom filter value exists then attempting to retrieve the combination from the KV database using the access key that is based on the combination.

[0138] FIG. 4 illustrates method 400 for inserting a key value (KV) pair to a separated database.

[0139] Method 400 may start by step 410 of receiving a request to insert the KV pair to the separated database. The request may be a PUT command. The separated database may include a log structured merge (LSM) tree and KV database that is separated from LSM tree.

[0140] Step 410 may be followed by step 420 of determining whether the KV pair should be associated with a KV pair bloom filter value.

[0141] Step 420 may include determining whether the KV pair should be associated with a versioned LSM entry or with a non-versions LSM entry. Step 420 may include step 320.

[0142] When determining that the KV pair should be associated with the KV pair bloom filter value then step 420 may be followed by step 430 of inserting the KV pair, a KV timestamp and the KV pair bloom filter value to the separated database.

[0143] Step 430 may include defining an access key to the KV database, wherein the access key is based on the combination.

[0144] When determining that the KV pair should not be associated with the KV pair bloom filter value then step 420 may be followed by step 440 of inserting the KV pair and the KV timestamp without inserting the KV pair bloom filter value.

[0145] Step 440 may include preventing from calculating the KV pair bloom filter value.

[0146] Step 440 may include defining an access key to the KV database, the access key is based on the key and not on the timestamp.

[0147] Either one of steps 430 and 440 may include storing the combination of the value and the KV timestamp in the KV database.

[0148] The LSM tree and bloom filters associated with the LSM tree may be stored in one or more local memory units (for example cached, stored in SSDs), and the KV database may be stored in one or more remote memory units.

[0149] Method 400 may include step 350 of performing one or more additional operations. The one or more additional operations may include a memory management operation (for example flushing, compaction) and/or responding to requests such as read requests.

[0150] FIG. 5 illustrates method 500 for managing a separated database.

[0151] Method 500 may start by step 510 of storing multiple values that belong to multiple KV pairs in the separated database. The separated database may include a log structured merge (LSM) tree and KV database that is separated from LSM tree.

[0152] One or more KV pairs may be associated with versioned LSM tree entries.

[0153] One or more other KV pairs may be associated with non-versioned LSM tree entries.

[0154] A current state of the separated database may be provided by responding to access requests, performing various management operations such as but not limited flushing, and compactions.

[0155] Step 510 may be followed by step 520 of finding that the given versioned LSM tree entry becomes a bottommost visible instance of a given KV pair within the LSM tree. This may be a result of compaction and a deletion of other visible versions of the given KV pair.

[0156] Step 520 may be followed by step 530 of converting the versioned LSM entry of the KV pair to a non-versioned entry of the KV pair. This may include deleting the KV pair bloom filter value that was associated with the versioned LSM entry.

[0157] FIG. 6 illustrates method 600 for managing a separated database.

[0158] Method 600 may start by step 610 of receiving a request to retrieve a given KV pair from a separated database. The given KV pair may include a given value and a given key. The separated database may include a log structured merge (LSM) tree and KV database that is separated from LSM tree. At a time of receiving the request, the

separated database stores multiple values that belong to multiple KV pairs in the separated database. One or more KV pairs are associated with KV pair bloom filter values and one or more other KV pairs are not associated with KV bloom filter values.

[0159] Step **610** may be followed by step **620** of checking whether a given KV pair bloom filter value exists.

[0160] If the given KV pair bloom filter value exists—step **620** is followed by step **630** of attempting to retrieve the at least given value from the KV database using an access key than is based on a given combination of the given key and a given KV pair timestamp.

[0161] If the given KV pair bloom filter value does not exist—step **620** is followed by step **640** of attempting to retrieve, from the KV database, at least the given value, using an access key that is based on the given key.

[0162] The following pseudocode provide various example for implementing one or more of the mentioned above methods.

[0163] The inventors implemented the method in Rock-SDB. Other implementations may be provided.

[0164] The pseudo code below may be a bit simplified.

[0165] A background job (flush/compaction) scheduler provides the following:

[0166] For method correctness, intra-flush concurrency is disabled (i.e., memtable flushes do not overlap in time).

[0167] To avoid technical complexity, L0→L0 (intra-level) compactions are disabled.

[0168] With proper system tuning, these constraints bear no performance penalty.

Flush/Compaction APIs

[0169] We assume that both flush and compaction are implemented as iterators that provide callbacks for each row (version) they scan. There are two types of callbacks: one for visible rows (i.e., the rows that some read may see), and one for invisible rows (i.e., the rows that no read can see). The implementation details of these iterators are unrelated to key-value separation. For example, an iterator may buffer multiple rows pertaining to the same key prior to deciding which of them are invisible.

[0170] We assume that all versions of each key are iterated through contiguously, in the descending order of timestamps (i.e., last-to-first).

Pseudocode Notation

```

Row types
// Generic LSM-tree row type, the rest derive from it
Struct Row { key, val, ts } // ts stands for "timestamp"
Struct MemtableRow extends Row { key, val, ts, type }
type ∈ { MT_PUT, MT_DELETE }
Struct SstRow extends Row { key, val, ts, type }
type ∈ { SST_PUT_V, SST_PUT_NV, SST_DELETE }
// External KV table entry
Struct Pair { key, val{ts, data} } // val is a concatenation of ts and data
Read path (GET and SCAN)
Public get(key, LSM, KV)
    pair ← get_from_snapshot(key, LSM.current_ts, LSM, KV)
    Return pair
Public get_from_snapshot(key, snap_ts, LSM, KV)
    // Fetch the latest version with timestamp ≤ snap_ts
    // LSM is using its Bloom filters
    row ← LSM.get_from_snapshot(key, snap_ts)
    If (row = ⊥) Then
        // Not covered by any BF but may still be in KV (NONVERSIONED reference)
        pair ← get_and_verify(key, snap_ts, KV)
    Else
        // No need to verify the timestamp, we know what we are looking for
        pair ← lookup_row(row, KV)
    Endif
    Return pair
// The same lookup_row method is used for processing a row in scan iterator
Private lookup_row(row, KV)
    If (row.type = SST_DELETE) Then
        // Tombstone - either single-row or range delete
        pair ← ⊥
    ElseIf (row.type = SST_PUT_NV) Then
        pair ← KV.get(row.key) // false positive
    Else // type = SST_PUT_V
        pair ← KV.get(row.key □ row.ts)
        If (pair = ⊥) Then
            // Fallback. Take care of the race condition -
            // a concurrent compaction might have renamed this KV pair
            pair ← get_and_verify(row.key, snap_ts, KV)
        Endif
    Endif
    Return pair
Private get_and_verify(key, snap_ts, KV)
    pair ← KV.get(key)
    // Verify the timestamp (embedded in the value)
    If (pair != ⊥ And pair.val.ts > snap_ts) Then // Does not belong to the snapshot
        pair ← ⊥
    Endif
    Return pair

```


-continued

```

Flush
Public flush_iterator_handle_emit(mtRow, LSM, KV, newSST)
  If (mtRow.type = MT_PUT) Then
    If (is_bottommost_visible(mtRow, LSM, 0)) Then
      KV.put(mtRow.key, {mtRow.ts, mtRow.val})
      newSST.append(mtRow.key, 1, mtRow.ts, SST_PUT_NV)
    Else
      KV.put(mtRow.key □ mtRow.ts, { 1, mtRow.val } )
      newSST.append(mtRow.key, 1, mtRow.ts, SST_PUT_V)
      newSST.BF.add(mtRow.key)
    Endif
  Else // mtRow.type = MT_DELETE
    newSST.append(mtRow.key, 1, mtRow.ts, SST_DELETE)
    newSST.BF.add(mtRow.key)
  Endif
Public flush_iterator_handle_skip(sstRow, LSM, KV, skip_type)
  // Redundant memtable entries are eliminated with no external effect.
Compaction
Public compaction_iterator_handle_emit(sstRow, LSM, KV, newSST)
  If (sstRow.type = SST_PUT_V)
    If (is_bottommost_visible(sstRow, LSM, newSST.level+1)) Then
      // Rename (VERSIONED → NONVERSIONED)
      v_key ← sstRow.key □ sstRow.ts
      pair ← KV.get(v_key)
      If (pair != 1) Then
        KV.put(sstRow.key, {sstRow.ts, pair.val.data})
        KV.delete(v_key)
      Else
        // Recovery - no work to do
      Endif
      sstRow.type ← SST_PUT_NV
    Else
      newSST.BF.add(sstRow.key)
    Endif
  ElseIf (sstRow.type = SST_PUT_NV) Then
    // Not much to do, already there. Just add to SST, no BF
  Else // (sstRow.type = SST_DELETE)
    newSST.BF.add(sstRow.key)
  Endif
  newSST.append(sstRow.key, 1, sstRow.ts, sstRow.type)
Public compaction_iterator_handle_skip(sstRow, LSM, KV, skip_type)
  If (sstRow.type = SST_PUT_V) Then
    // Eliminate this entry - no sharing, no one can see it
    KV.delete(sstRow.key □ sstRow.ts)
  ElseIf (sstRow.type = SST_PUT_NV)
    If (skip_type = SKIP_DELETE) Then
      // The same compaction observed a previous (newer) tombstone.
      // Delete the object, unless already overwritten by a previous compaction
      pair ← KV.get(key) // We can prove that get( ) .. delete( ) is atomic
      If (pair != 1 And pair.val.ts = sstRow.ts) Then
        KV.delete(sstRow.key)
      Endif
    Else // skip_type = SKIP_OVERWRITE
      // Optimization - a previous (newer) PUT performed the rename
      // No work to do
    Endif
  Else // (sstRow.type = SST_DELETE)
    // Do nothing for tombstones. KV cleanup is handled thru PUT rows
  Endif
Service Functions
// Predicate returns true if there are no visible versions of this row's key at bigger LSM levels
// Conservative - false negatives are allowed, but false positives are not.
Private is_bottommost_visible(row, LSM, min_level)
  If (row.ts > LSM.snapshot_timestamps( ).min( )) Then
    Return False
  Endif
  For 1 in (min_level .. LSM.max_level( ))
    For SST in LSM.sst_files(1)
      If (SST.BF.contains(row.key)
        // There is an earlier VERSIONED reference to this key
      Or
        SST.RangeDeletes.covers(row.key)

```

-continued

```

    // Range delete is like a tombstone
) Then
    Return False
End
End
End
// No match found - this key version is the bottommost visible
Return True

```

[0171] In the foregoing detailed description, numerous specific details are set forth in order to provide a thorough understanding of the invention. However, it will be understood by those skilled in the art that the present invention may be practiced without these specific details. In other instances, well-known methods, procedures, and components have not been described in detail so as not to obscure the present invention.

[0172] The subject matter regarded as the invention is particularly pointed out and distinctly claimed in the concluding portion of the specification. The invention, however, both as to organization and method of operation, together with objects, features, and advantages thereof, may best be understood by reference to the following detailed description when read with the accompanying drawings.

[0173] It will be appreciated that for simplicity and clarity of illustration, elements shown in the figures have not necessarily been drawn to scale. For example, the dimensions of some of the elements may be exaggerated relative to other elements for clarity. Further, where considered appropriate, reference numerals may be repeated among the figures to indicate corresponding or analogous elements.

[0174] Because the illustrated embodiments of the present invention may for the most part, be implemented using electronic components and circuits known to those skilled in the art, details will not be explained in any greater extent than that considered necessary as illustrated above, for the understanding and appreciation of the underlying concepts of the present invention and in order not to obfuscate or distract from the teachings of the present invention.

[0175] Any reference in the specification to a method should be applied mutatis mutandis to a system capable of executing the method and should be applied mutatis mutandis to a non-transitory computer readable medium that stores instructions that once executed by a computer result in the execution of the method.

[0176] Any reference in the specification to a system should be applied mutatis mutandis to a method that may be executed by the system and should be applied mutatis mutandis to a non-transitory computer readable medium that stores instructions that may be executed by the system.

[0177] Any reference in the specification to a non-transitory computer readable medium should be applied mutatis mutandis to a system capable of executing the instructions stored in the non-transitory computer readable medium and should be applied mutatis mutandis to method that may be executed by a computer that reads the instructions stored in the non-transitory computer readable medium.

[0178] The invention may also be implemented in a computer program for running on a computer system, at least including code portions for performing steps of a method according to the invention when run on a programmable apparatus, such as a computer system or enabling a pro-

grammable apparatus to perform functions of a device or system according to the invention. The computer program may cause the storage system to allocate disk drives to disk drive groups.

[0179] A computer program is a list of instructions such as a particular application program and/or an operating system. The computer program may for instance include one or more of: a subroutine, a function, a procedure, an object method, an object implementation, an executable application, an applet, a servlet, a source code, an object code, a shared library/dynamic load library and/or other sequence of instructions designed for execution on a computer system.

[0180] The computer program may be stored internally on a non-transitory computer readable medium. All or some of the computer program may be provided on computer readable media permanently, removably or remotely coupled to an information processing system. The computer readable media may include, for example and without limitation, any number of the following: magnetic storage media including disk and tape storage media; optical storage media such as compact disk media (e.g., CD-ROM, CD-R, etc.) and digital video disk storage media; nonvolatile memory storage media including semiconductor-based memory units such as flash memory, EEPROM, EPROM, ROM; ferromagnetic digital memories; MRAM; volatile storage media including registers, buffers or caches, main memory, RAM, etc.

[0181] A computer process typically includes an executing (running) program or portion of a program, current program values and state information, and the resources used by the operating system to manage the execution of the process. An operating system (OS) is the software that manages the sharing of the resources of a computer and provides programmers with an interface used to access those resources. An operating system processes system data and user input, and responds by allocating and managing tasks and internal system resources as a service to users and programs of the system.

[0182] The computer system may for instance include at least one processing unit, associated memory and a number of input/output (I/O) devices. When executing the computer program, the computer system processes information according to the computer program and produces resultant output information via I/O devices.

[0183] In the foregoing specification, the invention has been described with reference to specific examples of embodiments of the invention. It will, however, be evident that various modifications and changes may be made therein without departing from the broader spirit and scope of the invention as set forth in the appended claims.

[0184] Moreover, the terms “front,” “back,” “top,” “bottom,” “over,” “under” and the like in the description and in the claims, if any, are used for descriptive purposes and not necessarily for describing permanent relative positions. It is

understood that the terms so used are interchangeable under appropriate circumstances such that the embodiments of the invention described herein are, for example, capable of operation in other orientations than those illustrated or otherwise described herein.

[0185] The connections as discussed herein may be any type of connection suitable to transfer signals from or to the respective nodes, units or devices, for example via intermediate devices. Accordingly, unless implied or stated otherwise, the connections may for example be direct connections or indirect connections. The connections may be illustrated or described in reference to being a single connection, a plurality of connections, unidirectional connections, or bidirectional connections. However, different embodiments may vary the implementation of the connections. For example, separate unidirectional connections may be used rather than bidirectional connections and vice versa. Also, plurality of connections may be replaced with a single connection that transfers multiple signals serially or in a time multiplexed manner. Likewise, single connections carrying multiple signals may be separated out into various different connections carrying subsets of these signals. Therefore, many options exist for transferring signals.

[0186] Although specific conductivity types or polarity of potentials have been described in the examples, it will be appreciated that conductivity types and polarities of potentials may be reversed.

[0187] Each signal described herein may be designed as positive or negative logic. In the case of a negative logic signal, the signal is active low where the logically true state corresponds to a logic level zero. In the case of a positive logic signal, the signal is active high where the logically true state corresponds to a logic level one. Note that any of the signals described herein may be designed as either negative or positive logic signals. Therefore, in alternate embodiments, those signals described as positive logic signals may be implemented as negative logic signals, and those signals described as negative logic signals may be implemented as positive logic signals.

[0188] Furthermore, the terms “assert” or “set” and “negate” (or “deassert” or “clear”) are used herein when referring to the rendering of a signal, status bit, or similar apparatus into its logically true or logically false state, respectively. If the logically true state is a logic level one, the logically false state is a logic level zero. And if the logically true state is a logic level zero, the logically false state is a logic level one.

[0189] Those skilled in the art will recognize that the boundaries between logic blocks are merely illustrative and that alternative embodiments may merge logic blocks or circuit elements or impose an alternate decomposition of functionality upon various logic blocks or circuit elements. Thus, it is to be understood that the architectures depicted herein are merely exemplary, and that in fact many other architectures may be implemented which achieve the same functionality.

[0190] Any arrangement of components to achieve the same functionality is effectively “associated” such that the desired functionality is achieved. Hence, any two components herein combined to achieve a particular functionality may be seen as “associated with” each other such that the desired functionality is achieved, irrespective of architectures or intermedial components. Likewise, any two components so associated can also be viewed as being “operably connected,” or “operably coupled,” to each other to achieve the desired functionality.

[0191] Furthermore, those skilled in the art will recognize that boundaries between the above described operations merely illustrative. The multiple operations may be combined into a single operation, a single operation may be distributed in additional operations and operations may be executed at least partially overlapping in time. Moreover, alternative embodiments may include multiple instances of a particular operation, and the order of operations may be altered in various other embodiments.

[0192] Also for example, in one embodiment, the illustrated examples may be implemented as circuitry located on a single integrated circuit or within a same device. Alternatively, the examples may be implemented as any number of separate integrated circuits or separate devices interconnected with each other in a suitable manner.

[0193] Also for example, the examples, or portions thereof, may implemented as soft or code representations of physical circuitry or of logical representations convertible into physical circuitry, such as in a hardware description language of any appropriate type.

[0194] Also, the invention is not limited to physical devices or units implemented in non-programmable hardware but can also be applied in programmable devices or units able to perform the desired device functions by operating in accordance with suitable program code, such as mainframes, minicomputers, servers, workstations, personal computers, notepads, personal digital assistants, electronic games, automotive and other embedded systems, cell phones and various other wireless devices, commonly denoted in this application as ‘computer systems’.

[0195] However, other modifications, variations and alternatives are also possible. The specifications and drawings are, accordingly, to be regarded in an illustrative rather than in a restrictive sense.

[0196] In the claims, any reference signs placed between parentheses shall not be construed as limiting the claim. The word ‘comprising’ does not exclude the presence of other elements or steps than those listed in a claim. Furthermore, the terms “a” or “an,” as used herein, are defined as one or more than one. Also, the use of introductory phrases such as “at least one” and “one or more” in the claims should not be construed to imply that the introduction of another claim element by the indefinite articles “a” or “an” limits any particular claim containing such introduced claim element to inventions containing only one such element, even when the same claim includes the introductory phrases “one or more” or “at least one” and indefinite articles such as “a” or “an.” The same holds true for the use of definite articles. Unless stated otherwise, terms such as “first” and “second” are used to arbitrarily distinguish between the elements such terms describe. Thus, these terms are not necessarily intended to indicate temporal or other prioritization of such elements. The mere fact that certain measures are recited in mutually different claims does not indicate that a combination of these measures cannot be used to advantage.

[0197] While certain features of the invention have been illustrated and described herein, many modifications, substitutions, changes, and equivalents will now occur to those of ordinary skill in the art. It is, therefore, to be understood that the appended claims are intended to cover all such modifications and changes as fall within the true spirit of the invention.

We claim:

1. A method for updating a key-value (KV) storage system, the method comprises:

receiving a request to write a KV key to a row cache of the KV storage system, wherein the request is associated with a row cache key (RCK), wherein the RCK comprises a row cache identifier (RCID) and a column family identifier (CFID), wherein the KV storage system further comprises a separated database, the separated database comprises a log structured merge (LSM) tree and KV database that is separated from LSM tree; and

invalidating, responsive to the reception of the request, any KV key already cached in the row cache and associated with a same RCK as the RCK associated with the request.

2. The method according to claim 1, wherein the invalidating is executed without evaluating a timing relationship between the KV key and any of the KV keys already cached in the row cache.

3. The method according to claim 1, wherein the RCK does not include a file identifier (FID) that is indicative of a LSM file associated with the KV key.

4. The method according to claim 1, wherein the KV key is a versioned KV key.

5. The method according to claim 1, wherein the KV key is a non-versioned KV key.

6. The method according to claim 1, further comprising updating the row cache with the KV key.

7. The method according to claim 6, wherein the updating is triggered by a reception, by the KV storage system, of a request to read the KV pair.

8. The method according to claim 6, wherein the updating is triggered by a reception, by the KV storage system, of a request to read the KV pair that resulted in a memory tables related miss, wherein the memory tables store the LSM tree.

9. The method according to claim 6, wherein the updating is triggered by a reception, by the KV storage system, of a request to read the KV pair that resulted in a memory tables related miss and in a row cache miss, wherein the memory tables store the LSM tree.

10. A non-transitory computer readable medium for updating a key-value (KV) storage system, the non-transitory computer readable medium stores instructions for:

receiving a request to write a KV key to a row cache of the KV storage system, wherein the request is associated with a row cache key (RCK), wherein the RCK comprises a row cache identifier (RCID) and a column family identifier (CFID), wherein the KV storage sys-

tem further comprises a separated database, the separated database comprises a log structured merge (LSM) tree and KV database that is separated from LSM tree; and

invalidating, responsive to the reception of the request, any KV key already cached in the row cache and associated with a same RCK as the RCK associated with the request.

11. The non-transitory computer readable medium according to claim 10, wherein the invalidating is executed without evaluating a timing relationship between the KV key and any of the KV keys already cached in the row cache.

12. The non-transitory computer readable medium according to claim 10, wherein the RCK does not include a file identifier (FID) that is indicative of a LSM file associated with the KV key.

13. The non-transitory computer readable medium according to claim 10, wherein the KV key is a versioned KV key.

14. The non-transitory computer readable medium according to claim 10, wherein the KV key is a non-versioned KV key.

15. The non-transitory computer readable medium according to claim 10, that further stores instructions for updating the row cache with the KV key.

16. The non-transitory computer readable medium according to claim 15, wherein the updating is triggered by a reception, by the KV storage system, of a request to read the KV pair.

17. The non-transitory computer readable medium according to claim 15, wherein the updating is triggered by a reception, by the KV storage system, of a request to read the KV pair that resulted in a memory tables related miss, wherein the memory tables store the LSM tree.

18. A key-value (KV) storage system, comprising:

a controller that is configured to receive a request to write a KV key to a row cache of the KV storage system, wherein the request is associated with a row cache key (RCK), wherein the RCK comprises a row cache identifier (RCID) and a column family identifier (CFID), wherein the KV storage system further comprises a separated database, the separated database comprises a log structured merge (LSM) tree and KV database that is separated from LSM tree; and

invalidate, responsive to the reception of the request, any KV key already cached in the row cache and associated with a same RCK as the RCK associated with the request.

* * * * *